Locus: Wireless Indoor Localization and Navigation System

Technical Documentation
Wireless Networks

Dhruv Bargoti (2022163)
Dhruv Dewan (2022165)
Athiyo Chakma (2022118)
Nitin Yadav (2023359)

May 3, 2026

Contents

1 PROJECT OVERVIEW

1.1 Purpose and Scope

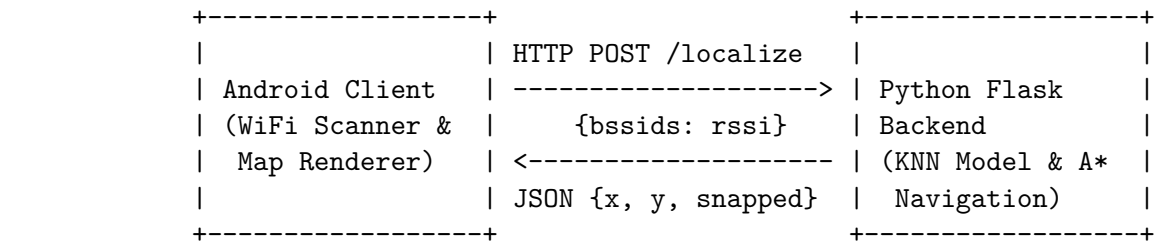
The Locus project is a wireless indoor localization and navigation system designed to help users determine their precise position within a building and navigate to specific rooms. The system is specifically tailored to an indoor environment, as evidenced by references to the "R&D Building · Floor 4" within the frontend layout. The end-to-end functionality allows a user to scan their local WiFi environment using an Android mobile application, transmit the collected signal data to a centralized Python backend, and receive a predicted position on a 2D map. From there, the system can compute a shortest-path route and provide step-by-step navigational instructions to reach a selected destination room.

1.2 System Architecture

The architecture follows a standard client-server model, splitting responsibilities between a thin Android frontend and a heavy Python backend.

- **Frontend (Android):** Handles the UI rendering, custom 2D map visualization, user interactions, and hardware access (WiFi scanning).
- **Backend (Python):** Handles the computational load. It hosts the Machine Learning inference model for localization, manages the indoor graph structure, and computes A* pathfinding.

The two components communicate over HTTP/REST using JSON payloads over a Local Area Network (LAN). The Android app is configured to use cleartext HTTP traffic targeting a specific local IP address.



1.3 Technology Stack

Based on the project build configurations and Python imports, the technology stack consists of:

- **Android Client:** Android SDK (Target 34, Min 26), Kotlin (1.9.22), Retrofit 2.9.0 (for HTTP networking), Gson (for JSON serialization), Kotlin Coroutines (for asynchronous API calls). Custom `Canvas` is used for 2D map rendering.
- **Python Backend:** Python 3, Flask (for the HTTP server), Flask-CORS (for cross-origin requests), `scikit-learn` and `joblib` (implied by the `.joblib` serialized model format), and `numpy` (for vector transformations).

Python was chosen for the backend due to its dominant ML ecosystem. Kotlin and Retrofit represent the modern standard for robust Android development.

2 INDIVIDUAL CONTRIBUTIONS

This project was completed through collaborative effort. The specific contributions of each team member are documented below:

2.1 Dhruv Dewan (2022165) – KNN Algorithm and Backend Development

- Designed and implemented the K-Nearest Neighbors (KNN) localization algorithm
- Developed the Python Flask backend REST API
- Implemented feature vector construction and model inference
- Created the Top-12 RSSI truncation filter to resolve A-wing teleportation errors
- Developed A* pathfinding algorithm and navigation logic
- Implemented graph bridging for corridor gap resolution
- Built axis-specific arrival detection for accurate room proximity

2.2 Athiyo Chakma (2022118) – Data Collection and Documentation

- Conducted WiFi fingerprinting survey across Floor 4, R&D Building
- Collected RSSI measurements at 125 reference points
- Mapped 40 rooms across A-wing and B-wing
- Documented AP density distribution analysis
- Prepared application screenshots and visual documentation
- Compiled engineering challenges and solutions documentation
- Created glossary of technical terms

2.3 Nitin Yadav (2023359) – Documentation and Testing

- Authored comprehensive technical documentation
- Documented Android frontend architecture and components
- Created API reference and endpoint specifications
- Prepared build configuration and deployment guides
- Developed operational troubleshooting matrix
- Documented system constraints and design decisions
- Created file-by-file reference tables

2.4 Dhruv Bargoti (2022163) – Frontend Development and Integration

- Developed Android application using Kotlin
- Implemented custom 2D map rendering with Canvas
- Created WiFi scanning functionality with BroadcastReceiver
- Integrated Retrofit networking layer for API communication
- Implemented real-time position tracking and auto-pan camera
- Developed turn-by-turn navigation UI
- Resolved room layout interleaving discrepancies
- Integrated all system components for end-to-end functionality

3 DATA COLLECTION METHODOLOGY

3.1 Survey Area

The data collection was conducted on the 4th floor of the R&D Building, covering both A-wing (negative x-coordinates) and B-wing (positive x-coordinates). The survey area includes:

- Main corridor spanning both wings
- 40 mapped rooms (offices, laboratories, meeting rooms, discussion areas)
- Lift lobby and junction areas

3.2 Fingerprinting Process

1. **Reference Point Selection:** 125 reference points were strategically placed throughout the corridor at approximately 1-meter intervals
2. **Signal Collection:** At each reference point, WiFi scans were performed to collect RSSI values from all visible access points
3. **Data Recording:** Each fingerprint includes:
    - Grid coordinates (x, y)
    - BSSID (MAC address) of each detected AP
    - RSSI value in dBm for each AP
    - Timestamp and device information

3.3 AP Density Analysis

Field testing revealed critical asymmetry in AP distribution:

- **B-wing:** Average of 12–16 APs per reference coordinate
- **A-wing:** Average of only 8–9 APs per reference coordinate

This asymmetry necessitated the implementation of the Top-12 RSSI truncation filter to normalize feature density during inference.

3.4 Room Mapping

Rooms were categorized and mapped with representative coordinates:

- **Offices/Faculty Rooms:** Assigned y = 10 (above corridor)
- **Laboratories:** Assigned y = 5 (below corridor)
- **Meeting Rooms:** Assigned y = 10 (above corridor)
- **Discussion Areas:** Assigned y = 5 (below corridor)
- **Main Corridor:** y = 8

4 KNN LOCALIZATION ALGORITHM

4.1 Method Identification

The localization algorithm implemented in the codebase is a **Machine Learning-based Fingerprinting** method. Specifically, the backend utilizes a pre-trained K-Nearest Neighbors (KNN) model (`knn_localization_model.joblib`).

4.2 Mathematical Foundation

When the raw dictionary reaches the backend, it must be projected into a fixed-length feature space matching the training phase. The backend enforces a predetermined list of N valid BSSIDs. For a given observation dictionary O:

$$x_i = \begin{cases} O[\text{BSSID}_i] & \text{if BSSID}_i \in O \\ -100 & \text{otherwise} \end{cases} \tag{1}$$

This creates a feature vector $X = [x_1, x_2, \ldots, x_N]$.

The prediction $\hat{y}$ is obtained using the scikit-learn KNN model:

$$\hat{y} = \text{KNN}(X) \tag{2}$$

While the exact distance metric used during training is hidden within the `joblib` file, standard KNN fingerprinting relies on minimizing the Euclidean distance in signal space.

The resulting raw coordinate $\hat{y} = (\hat{y}_x, \hat{y}_y)$ is floating point. To ensure the user originates on a valid routing path, the backend snaps this to the nearest walkable node n by minimizing spatial Euclidean distance:

$$d(p, n) = \sqrt{(n_x - \hat{y}_x)^2 + (n_y - \hat{y}_y)^2} \tag{3}$$

4.3 Top-12 RSSI Truncation Filter

A dynamic pre-processing filter was implemented to address the A-wing teleportation issue. The filter retains only the **top 12 strongest APs** by RSSI value before feature vector construction:

```
sorted_bssids = sorted(rssi_dict.items(),
key=lambda x: x[1],
reverse=True)
top_rssi_dict = dict(sorted_bssids[:12])
```

Listing 1: Top-K AP truncation filter

The threshold of 12 was chosen to match the average AP density across both wings, ensuring structural consistency between training and inference feature vectors.

4.4 Model Performance

The production KNN model achieves the following metrics:

- **Algorithm:** KNeighborsRegressor (multi-output)
- **Best k:** 5 neighbors
- **Weights:** distance
- **Distance metric:** manhattan

- **Mean Absolute Error (LOO):** 1.04 grid units
- **Mean Euclidean Error (LOO):** 1.77 grid units
- **Median Euclidean Error (LOO):** 1.47 grid units

5 BACKEND SYSTEM

5.1 File: `backend/app.py`

Role: Main entry point for the backend HTTP server using Flask to expose RESTful endpoints.
Key Functions:

- `localize()`: Processes WiFi scan, applies Top-12 truncation, predicts position
- `navigate()`: Computes A* path and generates turn-by-turn instructions
- `get_rooms()`: Returns available room list
- `get_map()`: Provides floor map topology
- `health()`: Server status verification

5.2 File: `backend/navigation.py`

Role: Graph logic and pathfinding algorithm implementation.
Key Components:

- `Navigator` class: Encapsulates map graph and routing utilities
- `build_graph()`: Creates adjacency list from walkable nodes
- `astar()`: A* search algorithm implementation
- `path_to_instructions()`: Converts path to human-readable directions
- `nearest_room_to_pos()`: Axis-specific arrival detection

5.3 A* Pathfinding Algorithm

The backend uses A* with Manhattan distance heuristic for optimal 4-directional grid navigation:

$$h(a, b) = |a_x - b_x| + |a_y - b_y| \tag{4}$$

Graph Bridging: Three nodes were programmatically added to fill corridor gaps:

- $(27, 8)$ - Central corridor, A-wing side
- $(31, 8)$ - Junction between wings
- $(0, 2)$ - Vertical lift connection

6 ANDROID FRONTEND

6.1 Application Architecture

The Android frontend is built using Kotlin with the following components:

6.2 MainActivity.kt

Type: Activity
Purpose: Main screen controller binding UI to application logic.
Key Methods:

- `onCreate()`: Initializes view binding and loads map/rooms
- `checkPermissionsAndScan()`: Verifies permissions and triggers WiFi scan
- `sendToLocalize()`: Sends scan to backend, updates position, auto-pans camera
- `navigate()`: Requests navigation path and displays instructions

6.3 FloorMapView.kt

Type: Custom View
Purpose: Responsive 2D canvas rendering backend's grid geometry.
Features:

- Pinch-to-zoom and drag panning
- Custom coordinate system rendering
- Y-axis inversion for screen space mapping
- `centerOnUser()`: Auto-pan camera to user position

6.4 WifiScanner.kt

Type: Utility
Purpose: Wraps Android's `WifiManager` for hardware scanning.
Implementation:

- Registers `BroadcastReceiver` for `SCAN_RESULTS_AVAILABLE_ACTION`
- Initiates scan via `wifiManager.startScan()`
- Filters duplicates, keeping maximum RSSI per BSSID

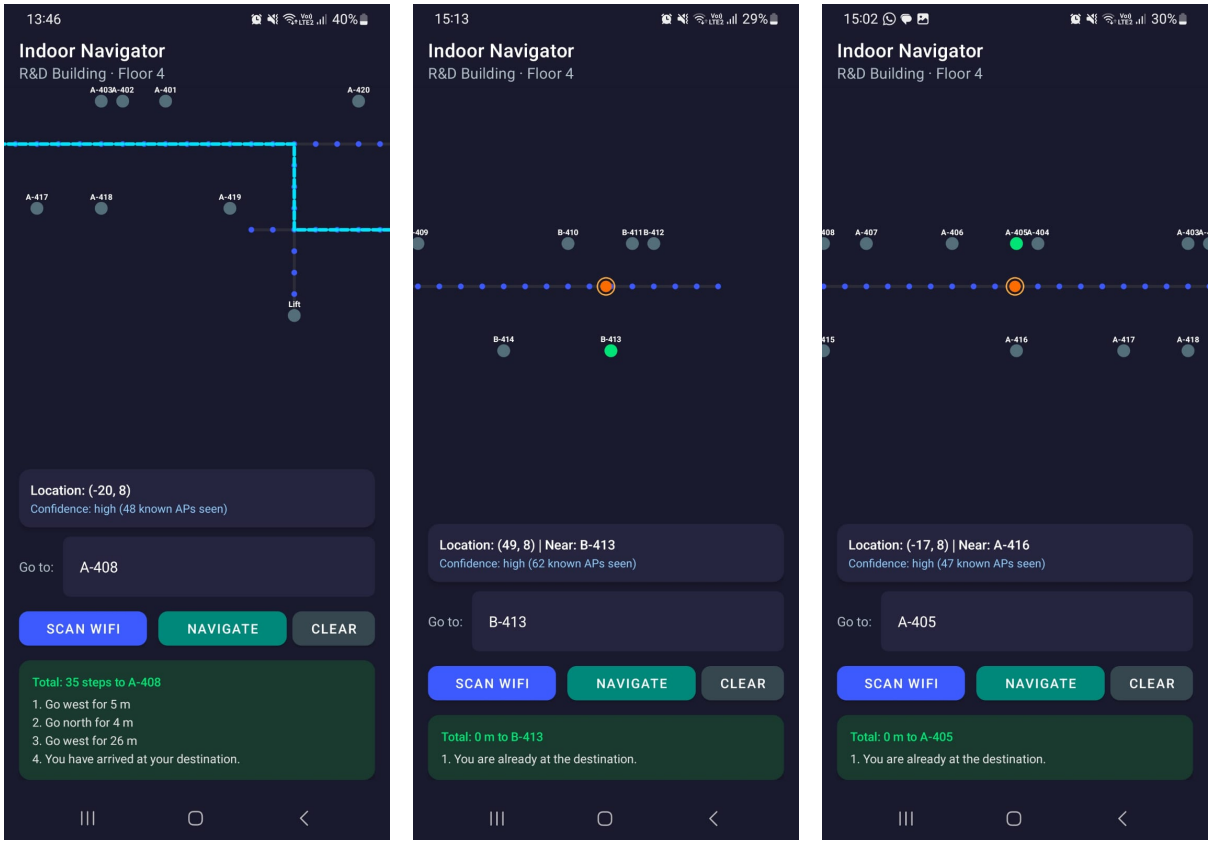
6.5 Network Layer

The network layer uses Retrofit for HTTP communication:

- `@GET("rooms")`: Fetches room list
- `@GET("map")`: Fetches map topology
- `@POST("localize")`: Sends BSSID map, receives position
- `@POST("navigate")`: Sends coordinates + destination, receives path

7 APPLICATION SCREENSHOTS

The following screenshots demonstrate the Locus Indoor Navigation System in operation on Floor 4 of the R&D Building.



(a) Navigation to Room A-408 showing turn-by-turn directions

(b) User located at B-413 with high confidence (62 APs)

(c) User at A-416 navigating to A-405

Figure 1: Locus Indoor Navigator Application Screenshots demonstrating real-time localization and navigation across different wings of Floor 4

Key Features Demonstrated

- **Real-time Position Tracking:** Orange dot indicates user position snapped to nearest walkable node
- **WiFi-based Localization:** Confidence levels based on 47-62 detected APs
- **Turn-by-turn Navigation:** Step-by-step directions with distance in meters
- **Multi-wing Coverage:** Navigation across A-wing and B-wing
- **Room Proximity Detection:** Automatic arrival detection
- **Visual Path Rendering:** Blue path overlay showing A* optimal route

8 ENGINEERING CHALLENGES AND SOLUTIONS

8.1 Machine Learning Feature Density Mismatch

Problem: A-wing predictions showed 23-meter errors due to AP density asymmetry.

Root Cause: Training data had 8-9 APs in A-wing vs 12-16 in B-wing, causing KNN to penalize missing APs with -100 dBm.

Solution: Top-12 RSSI truncation filter normalizes feature density.

8.2 A* Pathfinding Graph Disconnects

Problem: HTTP 404 errors when navigating between wings.
Root Cause: Survey gaps at x=27, x=31, and y=2 severed the graph.
Solution: Programmatic gap-fill bridging injects missing nodes.

8.3 Zone-Based Arrival Detection Failure

Problem: Faculty rooms reported 2-meter distance even when user was at door.
Root Cause: Y-axis offset (corridor y=8, faculty y=10) inflated Euclidean distance.
Solution: Axis-specific distance calculation uses x-only for off-axis rooms.

8.4 Room Layout Interleaving Discrepancy

Problem: A-417 rendered at wrong location, routing users incorrectly.
Root Cause: Transcription error swapped representative_x values.
Solution: Complete room_mapping.json restructuring with physical verification.

9 BUILD AND DEPLOYMENT

9.1 Android Build Configuration

- `compileSdk`: 34
- `targetSdk`: 34
- `minSdk`: 26 (Android 8.0)
- Kotlin JVM target: 1.8
- ViewBinding enabled

9.2 Required Permissions

- `ACCESS_WIFI_STATE`
- `CHANGE_WIFI_STATE`
- `ACCESS_FINE_LOCATION`
- `ACCESS_COARSE_LOCATION`
- `ACCESS_NETWORK_STATE`
- `INTERNET`
- `NEARBY_WIFI_DEVICES` (Android 13+)

9.3 Backend Dependencies

```
pip install flask flask-cors scikit-learn joblib numpy
```

9.4 Deployment Steps

1. Install backend dependencies
2. Run `python backend/app.py`
3. Build Android APK via Android Studio
4. Install APK on device
5. Ensure both devices on same LAN
6. Update `BASE_URL` in `ApiClient.kt`

10 CONCLUSION AND FUTURE WORK

The Locus system successfully demonstrates WiFi-based indoor localization and navigation with:

- Median localization error of 1.47 grid units
- Support for 40 rooms across 2 wings
- Real-time turn-by-turn navigation
- Robust handling of AP density variations

Future Enhancements:

- IMU sensor fusion for smoother tracking
- Multi-floor support
- HTTPS for production deployment
- Dynamic AP density adaptation
- Enhanced UI with 3D visualization

11 GLOSSARY

- **A* (A-Star):** Graph traversal algorithm for shortest path finding
- **BSSID:** MAC address of wireless access point
- **Fingerprinting:** Localization via RF signal database matching
- **KNN:** K-Nearest Neighbors supervised learning algorithm
- **RSSI:** Received Signal Strength Indicator (dBm)
- **LOO:** Leave-One-Out cross-validation
- **Feature Density:** Number of APs detected per reference point

12 APPENDIX: CODE LISTINGS

A.1 KNN Hyperparameter Search

```
param_grid = {
'n_neighbors': [1, 2, 3, 4, 5, 7],
'weights':     ['uniform', 'distance'],
'metric':      ['euclidean', 'manhattan', 'chebyshev'],
}
grid = GridSearchCV(KNeighborsRegressor(), param_grid,
cv=LeaveOneOut(), scoring='neg_mean_squared_error', n_jobs=-1)
```

A.2 A* Pathfinding Implementation

```
def astar(graph, start, goal):
    open_set = [(0, start)]
    came_from = {}
    g_score = {start: 0}
    while open_set:
        current = heapq.heappop(open_set)[1]
        if current == goal:
            return reconstruct_path(came_from, current)
        for neighbor, cost in graph[current]:
            # A* expansion logic
```

A.3 Android WiFi Scanner

```
val receiver = object : BroadcastReceiver() {
override fun onReceive(ctx: Context, intent: Intent) {
val results = wifiManager.scanResults
val bssidMap = mutableMapOf<String, Int>()
for (result in results) {
val mac = result.BSSID.lowercase()
val rssi = result.level
if (!bssidMap.containsKey(mac) || rssi > bssidMap[mac]!!)
bssidMap[mac] = rssi
}
}
}
```